

Optional<T>
Aufgaben

Average Optional

Schreibe eine Methode `calculateAverage`. Diese Methode akzeptiert eine Collection von Ganzzahlen als Parameter und gibt den Durchschnitt, sofern möglich, aller enthaltenden Zahlen zurück.

```
static Optional<Double> calculateAverage(Collection<Integer> numbers)
```

Falls die übergebende Collection leer ist, soll stattdessen ein leeres Optional zurückgeben werden.

Rufe die Methode für beide Fälle (leer und nicht leer) in einer main-Methode auf und gebe beide Ergebnisse in der Konsole aus.

Average Optional: Lösung

```
class AverageOptional {  
  
    static Optional<Double> calculateAverage(Collection<Integer> numbers) {  
        if (numbers.isEmpty()) {  
            return Optional.empty();  
        }  
  
        double sum = 0;  
        for (int num : numbers) {  
            sum += num;  
        }  
        double average = sum / numbers.size();  
        return Optional.of(average);  
    }  
  
    static void main(String[] args) {  
        var numbers = List.of(10, 20, 30);  
        Optional<Double> average = calculateAverage(numbers);  
        System.out.println(average.orElse(0.0));  
  
        Optional<Double> emptyAverage = calculateAverage(List.of());  
        System.out.println(emptyAverage.orElse(0.0));  
    }  
}
```

```
// Alternative mit Stream:  
static Optional<Double> calculateAverage(Collection<Integer> numbers) {  
    return numbers.stream()  
        .mapToInt(number -> number)  
        .average()  
        .stream().boxed().findFirst(); // convert OptionalDouble to Optional<Double>  
}
```

Find Student ID

Implementiere die folgende Methode:

```
record Student(int id, String name) { }
```

```
static Optional<Student> findByName(Collection<Student> students, String studentName)
```

Die Methode sucht in der übergebenen Collection students nach einem Student-Objekt, dessen Name mit dem übergebenen studentName übereinstimmt (ignoriere Groß- und Kleinschreibung).

Es wird der **erste Student** zurückgegeben, der gefunden werden kann.

Rufe die Methode in einer main-Methode auf. Gebe die id des gefundenen Student-Objekts in der Konsole aus, oder, falls nicht vorhanden "Student id not found."

Find Student ID: Lösung

```
class StudentFinder {  
  
    static Optional<Student> findByName(Collection<Student> students, String studentName) {  
        return students.stream()  
            .filter(student -> student.name().equalsIgnoreCase(studentName))  
            .findFirst();  
    }  
  
    static void main(String[] args) {  
        var students = List.of(  
            new Student(1, "Elon"),  
            new Student(2, "Anna")  
        );  
        Optional<Student> student = findByName(students, "elon");  
        System.out.println(getIdOrDefault(student));  
    }  
  
    private static String getIdOrDefault(Optional<Student> student) {  
        return student.map(s -> String.valueOf(s.id())).orElse("Student id not found.");  
    }  
}
```

Hinweis:

Optional<T> sollte nur für Rückgabewerte verwendet werden, aber nicht für Parameter!

Hier ist eine *Ausnahme*, weil die Hilfsmethode getIdOrDefault die Lesbarkeit in main verbessert.

Achtung:

student.orElse("Student id not found") würde hier nicht funktionieren, da orElse nicht die id sondern das Student-Objekt zurückgibt!

Statt map (aus Optional) könnte man hier auch if(student.isPresent()) {...} else {...} verwenden. map ist allerdings eleganter.

Finde Duplikate

Folgende Methode gibt alle Worte zurück, die in der übergebenen Collection doppelt vorkommen:

```
static ??? findDuplicates(Collection<String> words)
```

Überlege dir einen geeigneten Rückgabetyp (was ist, wenn keine Duplikate gefunden werden?).

Implementiere die Methode. Rufe die Methode in einer main-Methode für beide Fälle (keine Duplikate und Duplikate vorhanden) auf und gebe jeweils das Ergebnis in der Konsole aus.

Finde Duplikate: Lösung

```
class DuplicateFinder {  
  
    private static Map<String, Integer> frequencyMap;  
  
    static Collection<String> findDuplicates(Collection<String> words) {  
        Set<String> result = new HashSet<>();  
        frequencyMap = new HashMap<>();  
  
        for (String word : words) {  
            frequencyMap.put(word, numberOfOccurrences(word) + 1);  
            if (duplicateDetected(word)) {  
                result.add(word);  
            }  
        }  
        return result;  
    }  
  
    private static Integer numberOfOccurrences(String word) {  
        return frequencyMap.getOrDefault(word, 0);  
    }  
  
    private static boolean duplicateDetected(String word) {  
        return frequencyMap.get(word) == 2;  
    }  
  
    static void main(String[] args) {  
        var withDuplicates = List.of("apple", "banana", "apple", "orange", "banana", "grape");  
        System.out.println(findDuplicates(withDuplicates));  
  
        var noDuplicates = List.of("apple", "banana", "orange", "grape");  
        System.out.println(findDuplicates(noDuplicates));  
    }  
}
```

Hinweis:

Wenn eine Collection oder Array als Rückgabetyp verwendet wird, reicht es i. d. R. eine leere Collection oder leeres Array zurückzugeben statt Optional<T> zu verwenden.